Expt. No. 9

Title: *Design the architecture and implement the autoencoder model for image compression*

Rubric	***Score (0 to 5)***
Knowledge	
Delivery	
Output and Conclusion	
Total	

Performed On: 22/09/2025

Sign: _____

Experiment 9

Aim: Design the architecture and implement the autoencoder model for image compression.

Software: Google Collab

Pre Lab Question:

Explain the use of autoencoders?

Autoencoders are used for data compression, noise reduction, and feature learning by encoding and reconstructing inputs.

Post Lab Question:

Explain different types of autoencoders?

Types of autoencoders include vanilla, denoising, sparse, and variational, each with different encoding or regularization strategies.

Theory: Autoencoders are a deep learning model for transforming data from a high-dimensional space to lower-dimensional space. They work by encoding the data, whatever its size, to a 1-D vector. This vector can then be decoded to reconstruct the original data (in this case, an image). The more accurate the autoencoder, the closer the generated data is to the original.

Autoencoders are a deep neural network model that can take in data, propagate it through a number of layers to condense and understand its structure, and finally generate that data again. In this tutorial we'll consider how this works for image data in particular. To accomplish this task an autoencoder uses two different types of networks. The first is called an **encoder**, and the other is the **decoder**. The decoder is just a reflection of the layers inside the encoder. Let's clarify how this works.

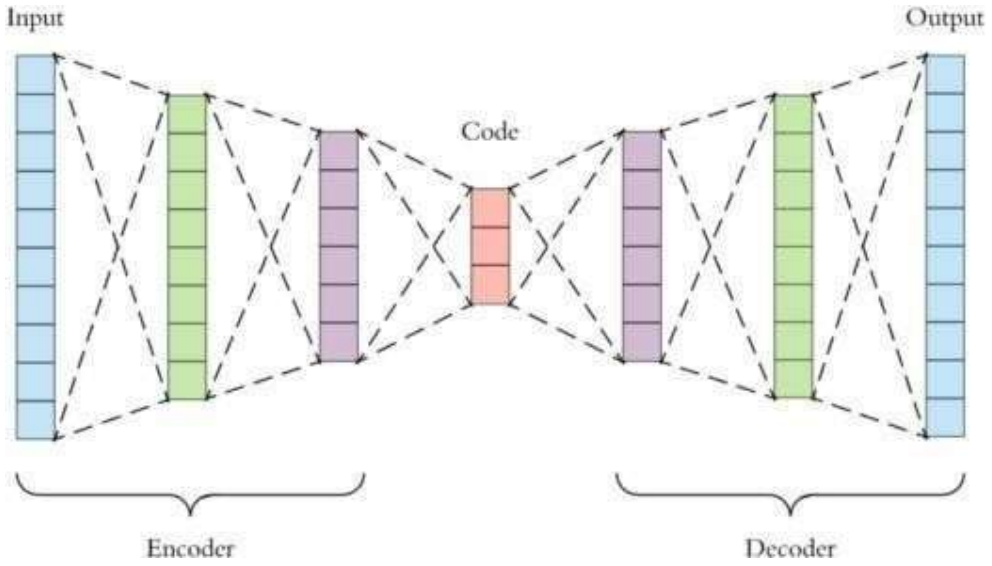
The job of the encoder is to accept the original data (e.g. an image) that could have two or more dimensions and generate a single 1-D vector that represents the entire image. The number of elements in the 1-D vector varies based on the task being solved. It could have 1 or more elements. The fewer elements in the vector, the more complexity in reproducing the original image accurately.

By representing the input image in a vector of relatively few elements, we actually compress the image.

Architecture of Autoencoders

An Autoencoder consist of three layers:

1. **Encoder**
2. **Code**
3. **Decoder**



- **Encoder:** This part of the network compresses the input into a **latent space representation**. The encoder layer **encodes** the input image as a compressed representation in a reduced dimension. The compressed image is the distorted version of the original image.
- **Code:** This part of the network represents the compressed input which is fed to the decoder.
- **Decoder:** This layer **decodes** the encoded image back to the original dimension. The decoded image is a lossy reconstruction of the original image and it is reconstructed from the latent space representation.

Program & Output:

```
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dense, Flatten, Reshape
from tensorflow.keras.optimizers import Adam

# Load and preprocess data
(x_train, _), (x_test, _) = mnist.load_data()
x_train = x_train.astype('float32') / 255.0
x_test  = x_test.astype('float32') / 255.0

# Flatten images
x_train = x_train.reshape((len(x_train), 28 * 28))
x_test  = x_test.reshape((len(x_test), 28 * 28))

# Define encoding dimension
encoding_dim = 64  # Compress to 64 features

# Define input layer
input_img = Input(shape=(784,))

# Encoder
encoded = Dense(128, activation='relu')(input_img)
encoded = Dense(encoding_dim, activation='relu')(encoded)

# Decoder
decoded = Dense(128, activation='relu')(encoded)
decoded = Dense(784, activation='sigmoid')(decoded)

# Autoencoder model
autoencoder = Model(input_img, decoded)

# Compile the model
autoencoder.compile(optimizer=Adam(), loss='binary_crossentropy')

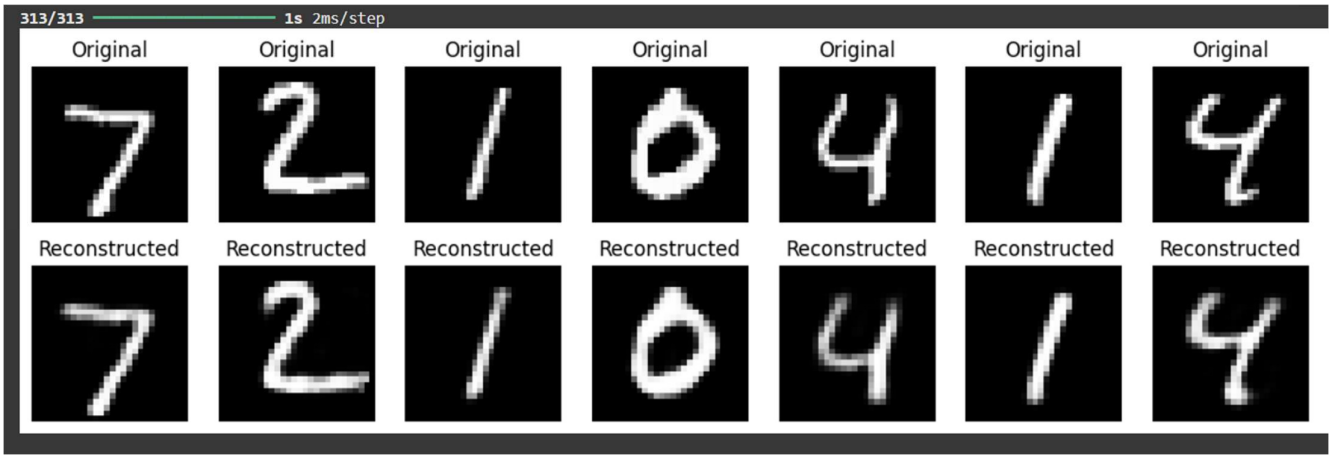
# Train the model
autoencoder.fit(x_train, x_train,
                epochs=20,
                batch_size=256,
                shuffle=True,
                validation_data=(x_test, x_test))
```

```
# Encode and decode test images
decoded_imgs = autoencoder.predict(x_test)

# Display original and reconstructed images
n = 10  # Number of digits to display
plt.figure(figsize=(20, 4))
for i in range(n):
    # Original
    ax = plt.subplot(2, n, i + 1)
    plt.imshow(x_test[i].reshape(28, 28), cmap='gray')
    plt.title("Original")
    plt.axis('off')

    # Reconstructed
    ax = plt.subplot(2, n, i + 1 + n)
    plt.imshow(decoded_imgs[i].reshape(28, 28), cmap='gray')
    plt.title("Reconstructed")
    plt.axis('off')
plt.show()
```

OUTPUT:



Conclusion:

Designing and implementing an autoencoder model for image compression enables efficient encoding of images into a lower-dimensional representation while preserving key features. This approach reduces storage needs and demonstrates how neural networks can learn compact, meaningful data representations through unsupervised learning.

CO's Covered:

CO2. Design and train feedforward neural networks using various learning algorithms and optimize model performance.
CO3. Build and train deep learning models such as Autoencoders, CNNs, RNN, LSTM, GRU etc.